AiWingman: A Local Continuity Overlay for Codex Task Portfolios with Retrospective Specification-Conformance Testing of Its Policy Layer

Mehmet Solak

Department of Biosystems Engineering, Faculty of Agriculture, Siirt University, Siirt, Türkiye
ORCID: 0000-0002-0800-0334 | Correspondence: mehmetsolak@siirt.edu.tr

Submission version 1.0 - 24 August 2026 - Technical Note manuscript

Abstract

Persistent coding-agent work can leave simultaneous input requests, completed results, blockers, and quiet unfinished tasks. AiWingman is an open-source macOS menu-bar companion that derives continuity cues from locally retained Codex task evidence. Its dashboard and optional Wingman review are separate; obsolete or abandoned labels require reversible user confirmation, not silence or age.

This Technical Note describes the system but evaluates only its pure continuity-policy layer, excluding both readers, the interface, graph signals, token accounting, Codex compatibility, and remote-review quality. The evaluation was retrospective and same-project: the Swift policy predated the written specification and 155 expected-output fixtures. The post-implementation specification and fixtures were frozen before the archived run, were not preregistered, and may reflect implementation knowledge; repository chronology does not exclude earlier exploratory or unarchived runs. A separately implemented same-project Python oracle generated expected outputs from the written rules. The Swift runner matched 155/155 fixtures. For each fixture, evaluations 2-100 matched evaluation 1 (15,500 evaluations; 15,345 nontrivial comparisons). A post-freeze supplementary driver launched 100 fresh processes on one arm64 Mac; all returned the same corpus digest. Across 70 unconfirmed lifecycle fixtures, none produced a confirmed-obsolete or confirmed-abandoned state. Later exact-commit CI reproduced both the conformance result and digest on hosted arm64 and native x86_64 macOS runners. The findings establish exact agreement with the 155 frozen same-project specification-derived fixture outputs, not general specification correctness. They do not establish external validation, human benefit, superiority, exact token cost or waste, security isolation, broad determinism, macOS 13 runtime compatibility, or stable Codex compatibility.

Keywords: coding agents; work continuity; local-first software; deterministic ranking suppression; specification-based testing; human oversight

1. Introduction

Coding-agent work is often organized as a portfolio rather than a single conversation. A user may delegate several tasks, answer one agent's question, leave another task waiting on an external event, receive a result in a third task, and later attempt to resume work after the original plan has faded. The newest timestamp is not necessarily the next useful task. Conversely, age and silence do not establish that work is obsolete or abandoned.

Prior work establishes programmer task context and resumption [1-4], awareness dashboards and personalized triage [5,6], agent-development and observability interfaces [7-11], and human oversight of multi-agent activity [12,13]. More recent trajectory and debugging systems add event replay, steering, annotation, and coding-agent visual analytics [14-19]. Current GitHub Copilot documentation describes parallel agent sessions, progress and token inspection, session-history queries, summaries, cost guidance, task resumption, deep links, and a separate critique agent [20-22]. Official OpenAI documentation likewise describes ChatGPT desktop Activity and goal workflows, goal workflows in Codex clients, code review across ChatGPT and Codex, and a Codex App Server interface for conversation history and streamed events [23-26]. These product-family precedents rule out first-of-kind claims for task portfolios, continuity, status display, session recall, review, token visibility, or agent-assisted critique. Because this report performs no direct comparative evaluation, it makes no superiority claim.

AiWingman's narrower contribution is an implementation-specific design case: an unofficial retrospective policy layer that combines locally retained Codex task evidence, an offline-default ordinary dashboard, explicit and reversible user-owned lifecycle decisions, and deterministic ranking suppression under written evidence rules. Its direct SQLite and deep-link adapter is an unsupported implementation choice and compatibility risk, not a novelty claim. Each constituent idea has prior art. The contribution is their auditable integration and the preparation of a versioned source and evaluation package.

AiWingman is not a coding agent, an autonomous project manager, an official OpenAI product, or a stable Codex integration. It observes local behavior that is not documented as a public compatibility contract. It writes AiWingman-owned preferences and continuity metadata but is not a write-free application. Its optional remote review is not part of the offline ordinary dashboard.

This report makes four bounded contributions:

1. It documents two distinct implementation pipelines without attributing task-tree aggregation or graph analysis to the ordinary dashboard.
2. It specifies a deterministic continuity policy that separates observed evidence from reversible lifecycle decisions and can return no recommendation under explicit conditions.
3. It reports a participant-free, specification-based evaluation of the pure policy layer, including exact conformance, within-process repeatability, a five-sentinel non-propagation check, a confirmed-state invariant, and descriptive-performance results; technical-baseline comparisons are labeled post-freeze exploratory contrasts, while later fresh-process and cross-architecture checks are identified as post-freeze supplementary evidence.
4. It provides a claim ledger that distinguishes historical release engineering evidence, current policy evidence, and reader, interface, remote, security, compatibility, and human-outcome questions outside the reported frozen policy benchmark.

Contributions 1 and 2 are source-backed system and policy descriptions. The frozen experiment evaluates only the pure policy layer in contribution 3; it does not evaluate either reader, the interface, the optional remote review, or human outcomes. Contribution 4 is an audit artifact rather than outcome evidence.

The reported evaluation recruited no participants and used no surveys, interviews, private task histories, or human-outcome measurements. The report does not claim faster resumption, improved recall, lower workload, better decisions, fewer wasted tokens, or increased productivity.

2. Related Work and Positioning

2.1 Programmer task context and resumption

Kersten and Murphy's Mylar work captured task context, filtered development artifacts by degree of interest, and restored context across task switches [1]. Parnin and DeLine investigated cues for resuming interrupted programming tasks through a survey and controlled study [2], while Parnin and Rugaber characterized resumption behavior across a large set of programming sessions [3]. These studies make generic task-context restoration and programming-resumption novelty untenable. They motivate AiWingman's checkpoints, next actions, and continuity cues, but they do not validate those features for coding-agent portfolios.

Research on user-authored source annotations also shows how developers use externalized cues for reminding and refinding [4]. AiWingman's next actions, waiting conditions, and lifecycle labels serve a related design function. Their usefulness in this product remains unmeasured.

2.2 Awareness dashboards and triage

Developer-awareness dashboards and feeds predate AiWingman [5]. Personalized issue-tracking views have likewise been studied as a way to reduce information overload [6]. These systems establish awareness and portfolio triage as prior concepts. AiWingman's policy is evaluated for agreement with its own frozen specification, not for relevance to a person's real work and not against these systems as a human-performance comparator.

2.3 Multi-agent interfaces, oversight, and observability

AutoGen Studio provides interfaces for building, running, evaluating, and debugging multi-agent workflows [7]. AgentScope, AgentBoard, and AgentOps offer multi-agent development, evaluation, monitoring, or observability abstractions [8-10]. AgentTrace proposes runtime instrumentation and structured operational, cognitive, and contextual traces [11]. AiWingman does not define or instrument an agent runtime; it retrospectively reads evidence retained by a local client.

Kitano et al. describe a dashboard for asynchronous review and human oversight of coordinating research agents [12]. Dhanorkar et al. identify a priori control, co-planning, real-time monitoring, and post-hoc review in interviews with experienced developers [13]. These are direct precedents for catch-up and oversight across agent activity. AiWingman's optional review is an implementation feature, not evidence that oversight quality improves. The Kitano workshop paper is cited by its workshop-hosted URL because the paper's displayed DOI is a placeholder and is not treated as a valid identifier.

Recent trajectory interfaces further narrow the contribution boundary. ReDel supports event-based logging and interactive replay [14]; AGDebugger combines a message-history overview with interactive reset and steering [15]; and Agent Trajectory Explorer supports trajectory visualization, annotation, and human feedback [16]. Coding-agent-specific visual analytics compare code evolution, solution processes, and model behavior [17], while AgentDiagnose and Graph of Trace provide structured trajectory inspection and visualization [18,19]. These systems primarily support runtime or post-hoc analysis of trajectories. They do not establish the usefulness of AiWingman's cross-session lifecycle decisions or deterministic ranking suppression.

Within the targeted primary-source comparison set reviewed on 23 August 2026, GitHub Copilot showed substantial documented overlap across several examined capabilities. This comparison was not a systematic review, does not rank all products, and does not establish novelty. Official Copilot documentation describes monitoring parallel sessions, viewing logs and token/session information, archiving, sharing, and resuming sessions, querying past sessions, producing stand-up summaries and cost or instruction-improvement suggestions, opening app deep links, and invoking a separate Rubber Duck critic [20-22].

Official OpenAI documentation is an even closer product-family precedent. It describes a ChatGPT desktop Activity view for unread, running, or waiting chats and status labels including Running, Needs input, Ready, and Blocked [23]; goal workflows with pause, resume, parallel chats, and status recaps across ChatGPT desktop and Codex clients [24]; stored-thread listing, status and history reads, archive and unarchive operations, and thread token-usage events through the Codex App Server [25]; and code review across ChatGPT and Codex, including a dedicated Codex reviewer and review pane [26]. These OpenAI product surfaces rule out feature-firstness for task portfolios, continuity, status display, review, or token visibility in this product family. AiWingman is therefore framed as a third-party retrospective policy case combining locally retained Codex evidence, reversible user-owned lifecycle metadata, and frozen deterministic ranking-suppression rules. Its direct database and deep-link integration is a compatibility risk rather than a novelty contribution.

2.4 Local-first design and deterministic ranking suppression

Local-first software emphasizes user control and continued operation without a service dependency [27]. Privacy engineering guidance treats data minimization, user participation, and transparency about collection and use as relevant design considerations [28]. AiWingman's ordinary dashboard adopts these as design principles; they are not inventions or privacy proofs. The optional remote path is excluded from the offline-default claim.

Classical reject-option and selective-classification research formalizes risk-error or risk-coverage relationships for predictors [29,30]. AiWingman is not a learned predictor, has no confidence calibration, and provides no statistical guarantee. It instead uses deterministic ranking suppression: fixed rules return no recommendation when eligible evidence is absent, the top score is below a threshold, or top candidates are too close. The term does not imply statistical selective prediction.

3. System Scope and Two Separate Pipelines

AiWingman is a native Swift application whose package declares a macOS 13 deployment target. The reported builds and tests ran on later macOS versions; no real macOS 13 launch or runtime result is reported. SwiftUI and AppKit provide the menu-bar interface. The source package separates reusable policy and reader components from the application interface, diagnostics, and self-tests. Selected executable, bundle, preference, and application-support identifiers retain the earlier Activity Radar name so upgrades preserve existing user-owned state.

The implementation has two distinct evidence pipelines. They share selected types and presentation surfaces, but they do not perform the same aggregation.

Pipeline	Input and reduction	Outputs and permitted claims
Ordinary dashboard	The `CodexActivityReader` queries candidate rows that are non-archived, have a nonempty preview, are absent from retained child-edge identifiers, and have a `thread_source` value of `user`, empty, or null. It reduces each selected candidate's own bounded rollout evidence and does not aggregate descendant rollouts into the root.	Per-candidate attention and continuity cues, explicit lifecycle controls, and a user-selected local deep link back to Codex. The observed-schema filters do not prove user ownership or complete root classification.
Optional Wingman analysis/review	The separate `CodexWingmanEvidenceReader` resolves parent-child task trees, scans bounded root and descendant rollout tails, and derives tree-level descriptive evidence.	Tree summaries, descriptive graph/theme signals, a maximum cumulative token-counter comparison proxy, and construction of a bounded packet for an optional consented CLI request.

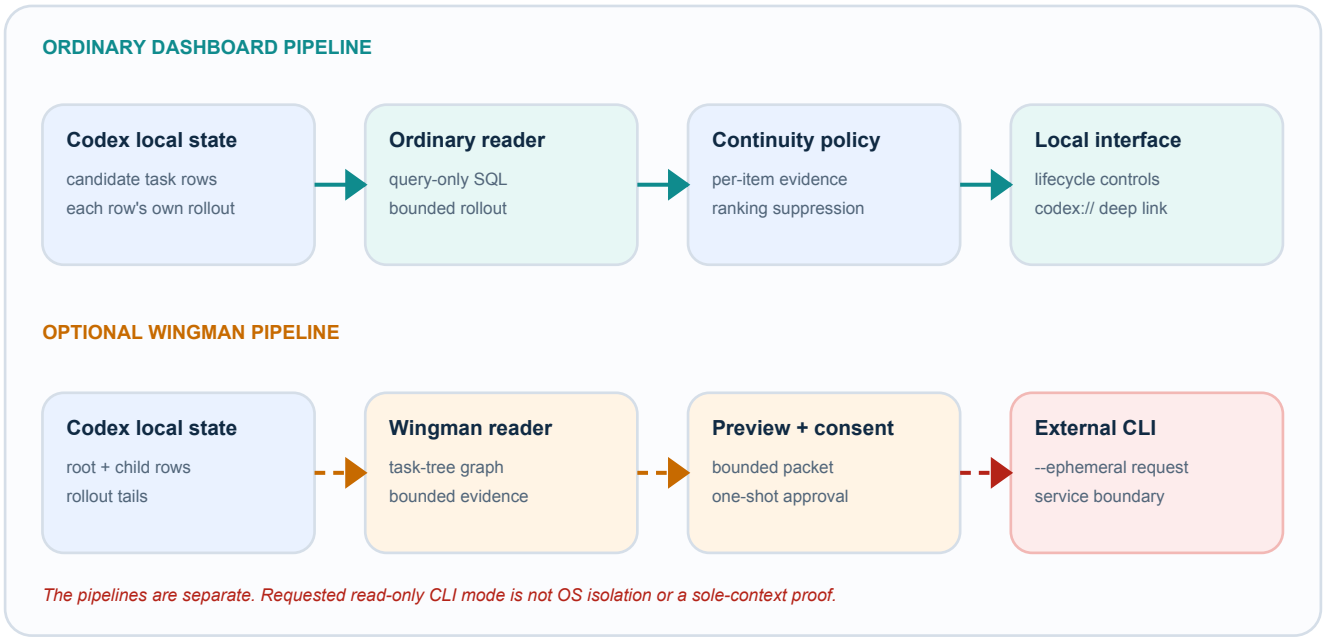

Figure 1. AiWingman's two pipelines and trust boundaries. The upper ordinary-dashboard path is local and per-candidate-row. The lower optional Wingman path constructs task trees and may cross a remote boundary only after packet preview and one-shot consent. The policy benchmark reported in this paper evaluates neither reader nor the remote path.

3.1 Ordinary dashboard

The ordinary path opens the local Codex SQLite database with `SQLITE_OPEN_READONLY` and `PRAGMA query_only=ON`. It queries candidate rows that are non-archived, have a nonempty preview, are not listed as children in retained spawn relationships, and have a `thread_source` value of `user`, empty, or null. The requested base page and total returned set are each capped at 200 rows. Priority and active/blocked/limited goal-linked inclusions are deduplicated and capped at 64. The inclusion scan takes at most 200 rows from a reverse-`rowid` goal window and orders eligible records from that window by `updated_at_ms`; it reports a conservative `hasMore` condition when the goal window or selected inclusions are truncated and then fetches goal metadata only for the bounded returned IDs. This is a finite compatibility window, not a guarantee that every retained goal is represented. Ordinary SQLite text is subject to per-field and aggregate byte ceilings before Swift string copying. These observed-schema filters do not prove user ownership or complete root classification. Each selected row's own rollout is sampled within fixed bounds and reduced to a local observation such as explicit input requested, unseen final result, blocked, recently active, quiet open work, or incomplete history. The dashboard combines these observations with AiWingman-owned continuity metadata and applies the pure policy layer described in Section 4.

These SQLite controls prevent AiWingman from issuing SQL statements that modify database content; they do not establish filesystem immutability. For a write-ahead-log-mode database, a read-only open is possible when the `-wal` and `-shm` files already exist and are readable, when the containing directory is writable so SQLite can create them, or when the database is opened as immutable [31]. Read-only SQL access therefore does not by itself establish filesystem immutability.

The ordinary dashboard may request that macOS open a user-selected task through a `codex://threads/<thread-id>` deep link. The implementation accepts only the standard hyphenated UUID layout, normalizes hexadecimal case, constructs it as one URL path segment, and rejects delimiter, traversal, braces, alternate layout, and encoded-separator inputs before handoff. A successful `NSWorkspace.open` return establishes only that macOS accepted the open request; AiWingman does not observe whether Codex displayed the requested task. The route itself is based on observed local behavior rather than a documented stable API. Unsupported schemas, absent files, unreadable evidence, or invalid identifiers are intended to fail neutrally instead of causing a source-state migration.

The current revision requires a standardized lexical descendant of the configured Codex root and opens each path component relative to that root with no-follow semantics. It rejects symbolic-link components and non-regular or untrusted files and bounds the session-index tail read. The SQLite adapter canonicalizes the parent path and requests a no-follow final open. These are property-specific implementation controls. They do not establish that every local input is safe or that the whole application is sandboxed. Reader robustness is outside the policy benchmark reported in Sections 5 and 6.

3.2 Optional Wingman analysis and review

The optional reader constructs a parent-child forest from retained spawn relationships, identifies roots and descendants, and samples bounded rollout tails across the tree. Duplicate edges are not intended to multiply evidence, and cycles or structurally inconsistent graphs are treated as compatibility failures. The tree reader is not used to make the ordinary dashboard's per-candidate-row continuity observation.

Spawned rollouts may share a cumulative token-counter lineage. Summing all counters in a tree can therefore double count shared history. The Wingman pipeline uses the largest observed cumulative counter in a tree as a tree-level cumulative comparison proxy. It is not a billed-token total, a period-specific quantity, a cost, a measure of usefulness, or a measure of waste. A date filter selects trees by recent activity; it does not turn a lifetime counter into usage during that date interval.

Text similarity, weighted degree, PageRank, and connected components are descriptive summaries of retained tree evidence. They are not measures of success, scientific interest, personality, cognitive load, or causal importance. They have no result in the frozen policy evaluation and are not part of the paper's core empirical claim.

3.3 User-owned state

Interface language, date range, and lifecycle choices are stored separately from Codex state. Continuity records pseudonymize task identifiers and are bounded by record and per-task history counts; the optional research ledger is additionally bounded by age and event count. The application therefore reads Codex state through adapters while writing its own state. It does not repair or migrate the Codex database.

4. Continuity Policy and Confirmation Contract

The frozen policy layer receives normalized observations and optional user-authored continuity metadata. It does not open databases, parse rollout files, construct graphs, invoke a model, or navigate the interface.

4.1 Precedence and deterministic scoring

Policy evaluation follows four stages:

1. Apply absolute deferrals.
2. Exclude an eligible item if its history is incomplete.
3. Derive ordered, fixed-vocabulary reason codes that contain no raw task text, together with integer weights.
4. Sort eligible candidates and apply deterministic ranking-suppression rules.

A future snooze is deferred as `snoozed`; a non-empty waiting condition is deferred as `waitingOnExternal`. Terminal work is also deferred when the frozen terminal conditions hold and no direct attention signal, near deadline, or due snooze overrides that state. Deferred items are not scored.

An incomplete, non-deferred item is excluded from scoring. Its presence does not suppress a recommendation supported by another complete item. If no candidate remains and an eligible incomplete item was observed, the policy returns `incompleteHistory` and exposes no ranking.

Table 1 gives the normative reason order and weights. The order is part of serialized output.

Order	Condition	Reason code	Weight
1	Explicit input requested	`explicitInput`	100
1	Blocked goal	`goalBlocked`	90
1	Unseen final result	`unseenResult`	80
1	Usage limited	`usageLimited`	70
1	Budget limited	`budgetLimited`	70
2	Deadline due or overdue	`deadlineOverdue`	65
2	Deadline within one day	`deadlineWithinDay`	55
2	Deadline within three days	`deadlineWithinThreeDays`	40
2	Deadline within seven days	`deadlineWithinWeek`	20
3	Planned return is due and no later accepted open request exists	`plannedReturnDue`	45

Order	Condition	Reason code	Weight
4	Critical importance	`criticalImportance`	30
4	High importance	`highImportance`	20
4	Low importance	`lowImportance`	-10
5	Non-empty next action	`nextActionRecorded`	10
5	No plan and age 7 to less than 30 days	`agingWithoutPlan`	12
5	No plan and age at least 30 days	`agingWithoutPlan`	18
6	Recently active execution	`recentlyActive`	8
7	An open request was accepted by macOS between 0 and 15 minutes ago, inclusive	`recentlyOpened`	-15

Candidates are sorted by descending score and then ascending activity identifier. A recommendation requires a score of at least 30 and, when a runner-up exists, a lead of at least 10. A top score below 30 returns `insufficientEvidence`; an existing runner-up less than 10 points behind returns `competingSignals`. When the policy returns no recommendation, it exposes no ranked candidates. A result with no candidates distinguishes `incompleteHistory`, `noEligibleWork`, and `insufficientEvidence` according to the frozen precedence.

The weights and the 30-point score and 10-point lead thresholds are author-designed constants inherited from the pre-existing implementation. They were not optimized against empirical user data, derived from participants, or calibrated as probabilities, confidence values, risks, or utilities. The frozen fixtures test their written behavior; they do not validate that these constants are appropriate for people.

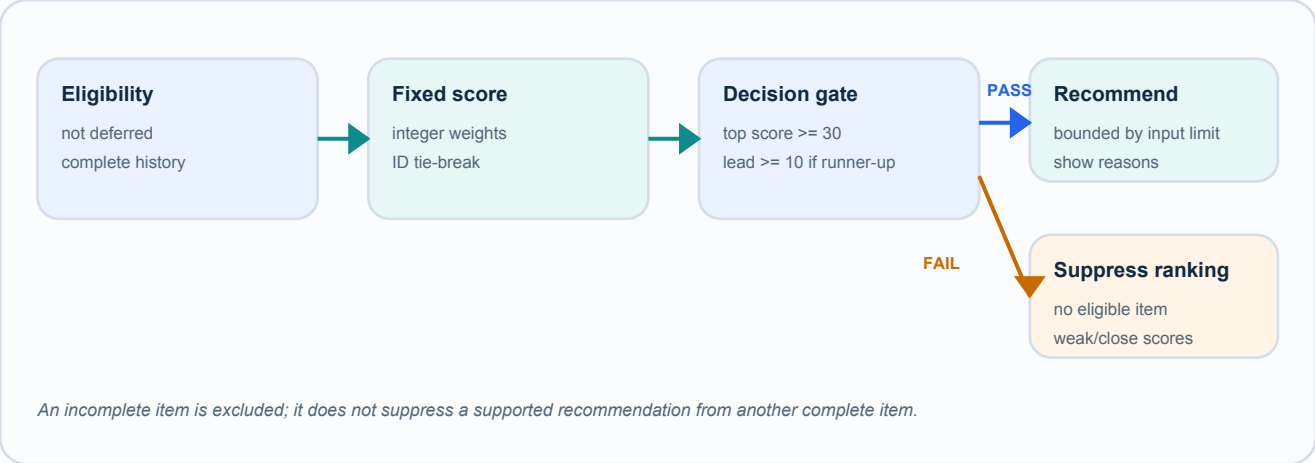

Figure 2. Frozen continuity-policy flow. Future snooze, waiting, and terminal rules defer an item; incomplete eligible history excludes that item; remaining complete items are scored; a score below 30 or, when a runner-up exists, a lead below 10 suppresses the ranking. An incomplete item does not suppress a recommendation from another complete item.

4.2 Lifecycle states

Observed evidence and lifecycle decisions are separate. A user confirmation is authoritative and maps directly to its reversible lifecycle state. Only explicit `abandoned` or `obsolete` confirmations can produce `abandonedConfirmed` or `obsoleteConfirmed`.

Without confirmation, the policy maps direct attention to `waitingHuman`, `blocked`, or `current`; an external waiting condition to `waitingExternal`; future snoozes, paused work, and sufficiently old quiet work to `dormant`; completed work to `completed`; and missing or unknown evidence to `uncertain`. Age can add inactive-evidence codes, but silence, age, and an aborted turn cannot infer a confirmed obsolete or abandoned state.

4.3 Optional remote-review boundary

The dashboard starts no background agent request. Opening the Wingman view also does not invoke Codex CLI. A separate explicit compatibility-check action validates the signed executable and runs version, command-compatibility, and login-status checks; it starts no agent turn and sends no AiWingman task packet, but it does create a private temporary Codex home with an opaque copy of the validated saved authentication file. The packet-carrying Wingman request is separately triggered. The intended user-derived packet is made available for inspection in a collapsed disclosure control before one-shot consent; the implementation verifies byte equality between that packet and the current preview but does not require the user to expand the control. The packet builder intentionally omits raw task identifiers, raw paths, configuration files, tool outputs, and authentication-file contents. The detailed portion can include sanitized titles for up to 12 task-tree rows; selected trees outside those detailed rows are represented through counts and aggregates. Sanitized prompt excerpts may be included only after an explicit opt-in. Human-authored continuity text is excluded. With prompt content disabled, task-derived free text is limited to sanitized titles; prompt excerpts, prompt-derived themes, local review signals, and next-move text are omitted. The packet still carries timestamps and the activity cutoff, status/enumeration fields, booleans, counts, numeric measurements, schema and response-language metadata, and a fixed method-boundary string.

Sanitization is best-effort and cannot prove removal of every secret, especially a secret embedded in a title or allowed text. Users must inspect the preview. Unexpected CLI JSONL items or tool events invalidate the returned review, but invalidation does not prove that the child process performed no earlier filesystem read, network exchange, or tool attempt. The child read-only mode is not an operating-system sandbox for the application.

The CLI is invoked with `--ephemeral`, which requests a turn intended not to save a local rollout; it does not prove that no local artifact exists and does not define service-side retention. The compatibility probe and review use private temporary roots whose names begin with the exact prefixes `ActivityRadar-CLI-Probe-` and `ActivityRadar-Wingman-`, respectively. A process-wide gate permits only one such remote operation at a time. Normal completion and error paths attempt removal and verify absence. If absence cannot be verified, the result is rejected, the unresolved root is latched, and later remote operations in the same application process retry cleanup and remain blocked while it fails. A crash or forced termination can bypass that path and leave a prefixed directory; restarting the application does not itself prove that residue was removed. These are implementation controls outside the frozen policy benchmark, not proof of security isolation or complete credential removal under every failure mode.

The ordinary dashboard contains no intended network path in the evaluated source architecture. Source-level checks for network APIs are bounded static evidence, not runtime non-interference. The optional CLI path intentionally reaches an external service after consent.

5. Evaluation Method

5.1 Scope and research questions

The evaluation covers only `WorkContinuityRanker` and `WorkContinuityLifecycle`. It excludes the ordinary reader, Wingman tree reader, Codex database and rollout compatibility, graph analysis, token proxies, interface behavior, deep links, diagnostics, local storage, optional CLI execution, remote output, and user outcomes.

The study asks four questions. RQ3 was formulated after the protocol and corpus freeze and is reported only as a post-freeze exploratory technical contrast:

- RQ1, conformance: Does the Swift policy match every expected output in the frozen synthetic corpus?
- RQ2, repeatability and bounded output checks: Do 100 within-process evaluations of each fixture yield one canonical output (that is, do evaluations 2-100 match evaluation 1), do the five exact seeded sentinels remain absent from the scanned serialized output surface, and do unconfirmed fixtures remain free of confirmed-obsolete or confirmed-abandoned states?
- RQ3, post-freeze exploratory technical comparisons: How often do two deliberately simplified rules produce the same selected identifier as the frozen policy, and how often do they select an item where the frozen policy requires no recommendation?
- RQ4, descriptive performance: What latency distribution is observed for synthetic portfolios of 10, 50, 200, and 1,000 inputs, and what single process-lifetime peak resident-memory value is observed after the full runner workload?

5.2 Freeze and chronology

The evaluation artifacts have a staged repository chronology in which the written specification and expected outputs were committed before the archived benchmark execution. The policy implementation already existed before this freeze, which was not a preregistration. Repository chronology and the author's account do not attest that no earlier exploratory or unarchived execution occurred.

Stage	Content-addressed checkpoint	Role
Historical superseded source release	`5e212181ae177cd555ab6bb92f5f71ac8be9173a`, tag `v1.2.0-beta.2`	Historical product and CI evidence; not the policy-result commit and not recommended for use
Protocol freeze	`53aa3e28862ff76092ad83d1347635fb1209a17d`	Written specification, Python generator, 155-fixture corpus, and freeze manifest; committed after policy implementation but before the archived benchmark execution
Swift runner	`fa5186e5d04fedfd75daac72e533a1daf9dbfa89`	Standalone benchmark executable; clean detached-worktree execution is author-reported rather than machine-attested by the result files
Result package	`e46686588793173d1b299b1829a92dbab7e9528d`	arm64 summary report plus the post-freeze 100-process supplementary script and summary

The specification SHA-256 is `ee6ab6568a3fcf8332facf9ef0a2d3bae66ab15d7f09a63ca6cbef6ce44e6f11`. The corpus SHA-256 is `183025163326aeb7f95470d2ecf494b31bc8239c8abb3e74e939967cf3ad8f30`, and the Python generator SHA-256 is `2c27f2bd7950e0aa26b8878ae27e13dfcd3332571f32b61444feb1111a8aae3c`. The freeze manifest records 75 triage fixtures and 80 lifecycle fixtures, for 155 total.

The evaluated policy source, `Sources/ActivityRadarCore/WorkContinuity.swift`, has SHA-256 `ff68e565bb1cebff47d66a23488e84b4e7bc2bf39cf8b75740683a7a055fa895` at the historical release, protocol-freeze, runner, result, and hardened-source checkpoints. Repository ancestry and these hashes show that the same source bytes were already present at the earlier release checkpoint and remained unchanged through the reported exact-commit CI checkpoint.

The author reports building and running the Swift checkpoint from a clean detached worktree. The V1 result package does not independently attest that procedure: the fresh-process driver's `--runner-commit` value is operator supplied, and the driver neither compares it with Git `HEAD`, requires a clean worktree, nor records the built executable's hash. The checkpoint association is therefore a provenance boundary to be verified through the revision, worktree-status, and source-hash checks in Section 9.

The Python standard-library generator does not import, invoke, parse, or copy Swift source. It implements the written specification as a separate oracle at the language boundary. However, the same project produced the specification, oracle, fixtures, and Swift implementation, and the specification and fixtures were authored after the implementation already existed. Knowledge of the implementation could therefore have shaped both the contract and its examples. This is a same-project, retrospectively specified, separately implemented oracle, not an implementation-independent reference standard or external validation.

5.3 Exact-output checks

The Swift runner strictly decodes the corpus, checks schema version and fixture counts, verifies the corpus bytes against the embedded corpus hash, and compares the corpus's embedded specification-hash value with the value compiled into the runner. It does not read or hash the specification file itself; the separate shell reproduction checks in Section 9 perform that file check. For triage fixtures it compares the recommendation or no-recommendation result, candidate identifiers and order, scores, ordered reason codes, weights, evidence times, and deferrals. For lifecycle fixtures it compares state, ordered evidence codes, evidence times, and ages. A fixture passes only if every specified field matches.

For the frozen within-process repeatability check, every fixture is evaluated 100 times. Evaluation 1 supplies that fixture's baseline canonical output; evaluations 2-100 are compared with it. The loop therefore performs 15,500 policy evaluations and 15,345 nontrivial equality comparisons (155 fixtures multiplied by 99 comparisons). Canonical JSON uses sorted keys, and one representative canonical output per fixture is framed with the frozen fixture identifier in frozen order to form the corpus digest. Timing, environment metadata, and expected outputs are excluded from the digest.

Post-freeze supplementary repeatability check. The frozen protocol specified the within-process repetition check but did not specify a 100-fresh-process driver. That driver and its result were added together in the result package. It launches the release benchmark 100 times. Each process checks conformance over all 155 fixtures and separately performs the runner's 100-evaluation-per-fixture repeatability loop before returning one canonical corpus-output digest. The supplementary evidence unit is therefore the process-level digest: 100/100 processes passed and returned one unique digest. It is not an additional 15,500-evaluation denominator and is not presented as preregistered evidence.

Result-package errata. The archived cross-process JSON contains `"evaluationCount": 15500`, computed by the V1 driver as 100 processes multiplied by 155 fixtures; `RESULTS_MANIFEST_V1.json` carries the analogous `crossProcessEvaluationCount`. Those labels describe process-fixture summary coverage, not individual policy-function calls. The within-process repeatability loops actually contain 1,550,000 policy evaluations across the 100 fresh processes (100 processes multiplied by 155 fixtures multiplied by 100 evaluations), including 1,534,500 nontrivial comparisons with per-fixture evaluation 1; these counts exclude separate conformance and performance calls. The historical `contentNeutrality` field name likewise denotes only absence of five

exact seeded strings from the scanned serialized output surface. This revision preserves the committed JSON files and hashes, records both terminology defects in `Research/results/RESULTS_ERRATA_V1.md`, and reports the narrower checks directly.

The five-sentinel non-propagation check uses five exact strings seeded in 10 field placements across nine fixtures, covering title, path, checkpoint, next-action, and waiting text. All 155 serialized conformance outputs were searched for all five strings. Outputs still contain activity identifiers, and the policy can depend on whether selected text fields are empty. The check therefore does not establish semantic content independence, anonymity, noninterference, or absence of identifiers. It also does not inspect either reader, the UI, logs outside the runner, diagnostics, preview packets, or remote requests. The confirmation-gated lifecycle invariant is checked over 70 fixtures without user confirmation; any `obsoleteConfirmed` or `abandonedConfirmed` output is a violation.

A retrospective observability audit found a narrower V1 coverage defect. When the policy abstains for a low score or close competition, it returns no ranked candidates. Consequently, no frozen expected output exposes the triage reason/weight pairs for `deadlineWithinWeek`, `lowImportance`, either `agingWithoutPlan` weight, or triage `recentlyActive`; the V1 triage corpus also has no zero-input portfolio. Changing one of those rules while preserving the same abstention could therefore leave 155/155 exact-output conformance unchanged. The V1 result remains exact agreement with its frozen outputs, not evidence that every written rule output was observed.

5.4 Post-freeze exploratory technical baselines

The frozen written protocol did not specify either baseline. Both contrasts first appeared in the later Swift runner commit, after the protocol and corpus freeze. They are therefore reported as post-freeze exploratory technical contrasts, not frozen or confirmatory evaluation components, competing products, or human-quality measures.

1. Recency-only always selects the input with the latest timeline activity, breaking ties by ascending identifier. It ignores deferrals, history completeness, metadata, score thresholds, and lead suppression.
2. Same-score/no-suppression applies the frozen deferrals, incomplete-history exclusion, weights, sorting, and identifier tie-break, but removes the 30-point threshold and 10-point lead requirement.

For each of the 75 triage fixtures, exact decision agreement means equality of the recommended activity identifier; `nil` means no recommendation. A no-recommendation violation occurs when a contrast selects an identifier for a fixture whose frozen expected result exposes no recommendation. This label does not imply that every such result arises from the score threshold or lead rule: the recency-only contrast also removes deferrals and history-completeness eligibility.

5.5 Post-freeze rule-observability supplement

The frozen V1 specification, corpus, and result artifacts were not modified. Six later engineering tests use combined inputs that make the four previously hidden triage reason codes, both `agingWithoutPlan` weights, and the zero-input abstention visible in exact assertions. A standard-library mutation harness copies the current Swift package to a temporary directory, confirms the six focused tests, then changes each asserted weight or empty-portfolio result one at a time. A mutation counts as killed only when the corresponding named test fails. This supplement is a retrospective regression check; it is not part of V1, a complete mutation analysis, exhaustive state coverage, an external oracle, or human-utility evidence.

5.6 Performance procedure and environment

The runner performs five warm-up calls followed by 30 measured repetitions for each input size. It reports the median, linearly interpolated p95, interquartile range, and maximum using `DispatchTime.uptimeNanoseconds`. Triage measures one ranking call over (N) synthetic inputs. Lifecycle measures (N) assessment calls over the same synthetic inputs. The 200-input series corresponds to the current ordinary-dashboard return cap; 1,000 inputs deliberately exceed that product limit and serve only as a synthetic policy-layer stress size. None of the series measures end-to-end dashboard latency.

The benchmark report machine-recorded arm64 architecture, macOS 26.6.2 build 25G83, 14 active processors, and release configuration. The separately authored result manifest additionally records a MacBook Pro model `Mac16,7`, Apple M4 Pro, 48 GB memory, Apple Swift 6.3.3, and target `arm64-apple-macosx26.0`; those additional fields are author-reported rather than runner-attested. Process peak resident memory was sampled after all series using macOS `getrusage(RUSAGE_SELF)`; it is a process-lifetime peak, not an allocation total or a per-operation measurement.

5.7 Generative-AI assistance in the research workflow

OpenAI Codex assisted with source discovery, code inspection, literature organization, drafting and revising the specification, Python oracle, synthetic fixtures, benchmark runner, tests, manuscript, and document formatting. Codex also assisted with author-directed internal adversarial reviews of claims, methods, security boundaries, and publication artifacts. These same-project reviews were not human or external peer review and were not independent validation. The named author is responsible for verifying every claim, reference, result, and generated artifact, revising the prose as needed, and approving the final submission. AI is not listed as an author and was not treated as an independent reviewer or independent validation source.

6. Results

6.1 Conformance, repeatability, and bounded contract checks

Table 2 summarizes the V1 machine-readable result report. Passing per-fixture actual outputs were not archived as rows; the artifact retains counts, failure records, and a corpus-output digest. It is therefore a summary report rather than a complete per-fixture result ledger. The primary report SHA-256 is `fa7e271a436be62bf9c7f9979122ccc6a01d36fe03abaea7b4a0277472a9957e`. The post-freeze fresh-process summary SHA-256 is `9d1dcd88b2dca1672ccfabee076ee2c9274ebb63458275b30e6a4269b37b4af1`.

Check	Denominator	Result	Supported interpretation
Frozen exact-output conformance	155 fixtures	155/155 passed	Swift policy matched all same-project specification-derived expected outputs
Frozen within-process repeatability	155 fixtures × 100 evaluations	Evaluations 2-100 matched evaluation 1 for every fixture; 15,500 total evaluations and 15,345 nontrivial equality comparisons	One representative output per fixture formed one corpus digest on the recorded process and machine
Post-freeze supplementary fresh-process repeatability	100 fresh processes	100/100 passed; one unique digest	Every process produced the same 155-fixture corpus digest on one arm64 machine after its internal repetition loop
Five-sentinel non-propagation	5 exact strings; 10 field placements across 9 fixtures; all 155 outputs searched for all 5 strings	0 exact sentinel occurrences	The five seeded strings did not appear in the scanned serialized policy-output surface
Confirmation-gated lifecycle invariant	70 unconfirmed fixtures	0 confirmed-state violations	No unconfirmed fixture produced confirmed obsolete or abandoned

The canonical corpus-output digest was `fd72e52b8098d2f62f6c6f7ccc6de7e744de7ed03722d08c7f983527601db8e6` in the primary run and all 100 supplementary fresh processes. The frozen conformance, within-process, sentinel, and lifecycle results answer RQ1 and RQ2 within the recorded arm64 environment. The fresh-process result is a later supplementary check. Neither result establishes external correctness, reader privacy, or remote-model repeatability.

6.2 Post-freeze rule-observability supplement

All six focused rule-observability tests passed on the current arm64 working revision. In a temporary package copy, five simple compiling weight mutations covered the four previously hidden reason codes, with separate mutations for the two `agingWithoutPlan` weights; a sixth mutation changed the empty-portfolio abstention result. Each corresponding focused test failed, so 6/6 enumerated mutations were killed. The machine-readable result is `Research/results/post-freeze-rule-observability-v1.json`; the runner is `Research/run_rule_observability_mutations.py`. The harness did not mutate the repository working tree. This evidence is post-freeze, same-project, narrow, and supplementary; it neither repairs the historical V1 corpus retroactively nor establishes exhaustive coverage.

6.3 Post-freeze exploratory technical baseline comparisons

Table 3 reports the post-freeze exploratory exact-decision agreement and no-recommendation violations over all 75 triage fixtures. These contrasts were not specified in the frozen written protocol.

Technical contrast	Exact agreement with frozen decision	No-recommendation violations
Recency-only	40/75 (53.3%)	35
Same-score/no-suppression	59/75 (78.7%)	16

The recency-only rule selected an item in all 35 frozen no-recommendation cases: 18 `insufficientEvidence`, 12 `noEligibleWork`, two `incompleteHistory`, and three `competingSignals` results. Because that contrast removes deferrals, history-completeness eligibility, scoring, and both suppression rules, its 35 cases locate the combined effect of those mechanisms rather than ranking suppression alone. The same-score/no-suppression contrast selected an item in 16 cases: 13 `insufficientEvidence` and three `competingSignals` results. Those 16 isolate the mechanical effect of removing the 30-point score threshold and 10-point lead requirement while retaining deferrals, eligibility, weights, sorting, and identifier tie-breaking. Fixture categories were deliberately constructed and were not sampled or prevalence-weighted. The agreement percentages are proportions within this synthetic contract corpus, not estimates of real-world error or recommendation frequency. Neither comparison shows that the frozen policy is more useful, accurate, efficient, or preferable for people.

6.4 Descriptive performance

Table 4 reports rounded summary statistics in milliseconds; the archived JSON retains the full recorded precision. Every cell is based on 30 measured repetitions after five warm-ups. The 1,000-input rows are synthetic policy-layer stress sizes above the ordinary dashboard's current 200-row return cap.

Operation	Inputs	Median (ms)	p95 (ms)	IQR (ms)	Maximum (ms)
Triage portfolio	10	0.00375	0.00386	0.000042	0.0125
Triage portfolio	50	0.0232	0.0241	0.000291	0.0396
Triage portfolio	200	0.0936	0.0948	0.000240	0.0951
Triage portfolio	1,000	0.511	0.567	0.0292	0.569
Lifecycle batch	10	0.00363	0.00369	0.000000	0.00383
Lifecycle batch	50	0.0190	0.0194	0.000115	0.0194
Lifecycle batch	200	0.0758	0.0949	0.00267	0.103
Lifecycle batch	1,000	0.388	0.425	0.0181	0.471

The process-lifetime peak resident memory after all performance series was 12,992,512 bytes, approximately 12.4 MiB. Because that value covers the entire runner process and all series, it cannot be attributed to a particular operation or input size. The latency and memory results are descriptive for synthetic policy inputs on one machine and cannot be converted into human time saved.

6.5 Release and exact-commit engineering evidence

GitHub Actions run `32648392604` evaluated the older public release commit `5e212181ae177cd555ab6bb92f5f71ac8be9173a` on 23 August 2026. The arm64 macOS 15 and native x86_64 macOS 15 jobs each reported 52 Swift tests and 16 deterministic self-tests passed. The Intel job built the release application; the arm64 gate also ran enumerated source, diagnostics, storage, manifest, and local Universal 2 package checks.

This is regression and packaging evidence only for the named release commit. It is separate from the arm64 policy benchmark at `fa5186e5...` and does not validate the later manuscript, protocol, reader-hardening, or result branch. The original mutable release-page copy and the release notes stored at commit `5e212181...` contained three documentation overstatements: they implied macOS 13 runtime support without a real macOS 13 run, stated that the dashboard never writes under `~/.codex` without the WAL `-shm` exception, and reduced the prompt-off preview packet to titles and numeric measurements. The unsigned annotated tag `v1.2.0-beta.2` currently resolves to that commit. The release-page body was corrected on 23 August 2026 with a post-tag notice; the notes stored at the historical commit remain unchanged. Those original statements are not treated as evidence here. A successful Swift command with zero discovered tests is not counted as evidence.

The beta2 source tag is historical and is not recommended for installation. Path containment, bounded session-index reads, and verified normal/error temporary-auth cleanup first appear at later hardened checkpoints and are outside the frozen policy benchmark.

A later exact-commit GitHub Actions run, `32659669054`, evaluated hardened source checkpoint `99faac3ef52d0da72d082706f64903a6aacd2c6d` on 23 August 2026. The GitHub-hosted macOS 15 arm64 and native x86_64 jobs each discovered and passed 82 Swift tests, passed 16 deterministic self-tests, and ran the 155-fixture benchmark. Both benchmark jobs returned 155/155 conformance and digest `fd72e52b8098d2f62f6c6f7ccc6de7e744de7ed03722d08c7f983527601db8e6`; the Intel job also built the release application, and the arm64 job passed the public-source preparation check and a synthetic diagnostic non-disclosure fixture. The policy source at this checkpoint is byte-identical to the source at the frozen runner checkpoint.

This exact-commit run is post-freeze engineering regression evidence on two hosted architectures. It is not part of the V1 result package, an independent reproduction, a broad cross-machine or operating-system claim, or a real macOS 13 runtime result.

7. Limitations and Threats to Validity

Retrospective same-project oracle. The written specification, Python oracle, fixtures, Swift implementation, and benchmark runner were produced within the same project, and the implementation predated the specification and fixture corpus. Language-boundary separation prevents direct source import but does not prevent knowledge of existing behavior from shaping the contract. Confirmation bias and shared misunderstandings can therefore survive 155/155 conformance.

Synthetic and finite coverage. The 155 fixtures instantiate a finite set of frozen expected outputs, not every state combination or every independently observable rule output. Because abstention hides ranked candidates, V1 does not expose

`deadlineWithinWeek`, `lowImportance`, either `agingWithoutPlan` weight, or triage `recentlyActive`, and it has no zero-input portfolio. The later six-test, six-mutation supplement closes only those enumerated observability gaps and is not retroactive V1 evidence or exhaustive mutation coverage. The non-propagation check covers five exact sentinels and only the scanned serialized policy-output surface. Outputs retain activity identifiers, and the policy can use text-field emptiness. The check is not semantic content independence, anonymity, general noninterference, privacy, or a secret-removal proof.

Policy-only scope. Neither Codex reader, graph construction, token proxy, UI, deep link, local storage, diagnostics, preview, child process, or remote response is part of the reported benchmark. Reader and remote-boundary tests must be reported separately and must not be added to the 155-fixture denominator.

Unreported gates. No separately frozen metamorphic ledger, complete adversarial reader ledger, or end-to-end remote disclosure ledger is reported in the V1 result package. Current-branch dual-architecture CI is reported separately as post-freeze engineering evidence. Unit tests and that CI run cannot be substituted for an omitted frozen ledger.

Core result and supplementary architecture scope. The frozen result and the 100-fresh-process supplementary check ran on one arm64 Mac. A later exact-commit CI run matched conformance and the digest on one hosted arm64 and one hosted x86_64 runner. That post-freeze run supports equality for those two jobs only; it is not external reproduction or broad cross-machine, compiler, or operating-system determinism.

Post-freeze exploratory baselines. The baselines were added after the protocol and corpus freeze and deliberately remove policy mechanisms. The 35 recency-only violations combine deferral, eligibility, scoring, threshold, and lead effects; only the 16 same-score/no-suppression violations isolate removal of the threshold and lead rules. These mechanical contrasts provide no external reference standard, user preference, or comparative superiority. The deliberately constructed fixture corpus is not a prevalence sample, so the agreement percentages are not real-world frequency estimates.

Performance scope. Timings use synthetic normalized inputs and exclude database I/O, rollout parsing, interface rendering, graph analysis, and remote calls. The 1,000-input case is above the current ordinary-dashboard return cap and is only a stress size. The process-level memory peak is not isolated by series. No independent process rerun, randomized series order, thermal or CPU-frequency control, or confidence interval was reported; each p95 is a descriptive linearly interpolated quantile of 30 observations.

Undocumented integration and SQLite boundary. AiWingman depends on observed Codex filenames, SQLite schemas, JSONL events, and deep-link routes that may change. Read-only/query-only opening prevents application SQL writes to database content. For a write-ahead-log-mode database, SQLite documents read-only opening when the `-wal` and `-shm` files already exist and are readable, when the containing directory is writable so SQLite can create them, or when the database is opened as immutable [31]. Neutral failure and logical read-only access cannot establish filesystem immutability or create a stable third-party API contract.

Token and graph interpretation. The optional tree maximum is a cumulative comparison proxy. It is neither period usage nor billed, useful, or wasted tokens. Graph summaries are descriptive and unevaluated.

Remote and security boundary. Packet minimization, preview, opt-in, event rejection, path checks, serialized temporary-auth use, and verified normal/error cleanup are narrow controls. Best-effort sanitization cannot guarantee secret removal. Output rejection does not prove absence of prior I/O, and a crash can leave a temporary root before cleanup or latch handling completes. The application is not claimed to be security-isolated, and no external security audit is reported.

No human-outcome evidence. No participants or private task histories were studied. The report cannot answer whether AiWingman improves recall, resumption time, workload, decision quality, productivity, or token use.

Archival and independence boundary. Hardened source checkpoint `99faac3...` is publicly reachable, and exact-commit run `32659669054` supplies bounded hosted CI evidence for that checkpoint. The V1 artifacts are versioned in the same project, but no independent scholarly reproduction or independent archival verification is reported. A later preprint DOI would document manuscript deposit, not validate the source, methods, or results.

8. Discussion

The evaluation supports a narrow conclusion: the pre-existing Swift continuity policy matched every expected output from a same-project synthetic specification committed before the archived benchmark execution, matched each fixture's first canonical output across the frozen within-process repetitions, omitted five exact seeded strings from the scanned serialized output surface, and respected the explicit-confirmation boundary in the tested lifecycle fixtures. Repository chronology does not exclude earlier exploratory or unarchived runs. A post-freeze fresh-process check and later dual-architecture CI returned the same digest within their stated environments. None of these checks establishes whether the policy chooses the right task for a person.

The V1 corpus does not expose every triage reason or the zero-input boundary. The later observability supplement makes the four hidden reason codes, both aging weights, and the empty-portfolio result explicit, and kills six corresponding simple mutations. This is useful current-branch regression evidence, but it does not alter the historical 155-fixture result or establish exhaustive coverage.

The post-freeze exploratory baseline results locate distinct mechanical effects. A pure recency rule selected an item in all 35 frozen no-recommendation cases while removing deferrals, eligibility, scoring, and suppression. Retaining the frozen score but removing only the threshold and lead rules selected an item in 16. The latter shows that the two ranking-suppression rules change synthetic contract decisions. Neither contrast establishes calibrated uncertainty or improved decisions.

The product-level design contribution is likewise integrative. Task context, reminders, dashboards, post-hoc oversight, and agent-session summaries are established. AiWingman combines selected versions of these ideas around locally retained Codex evidence and keeps three distinctions explicit: ordinary per-candidate-row inspection versus optional task-tree analysis; observed evidence versus user-owned lifecycle confirmation; and offline-default local analysis versus a previewed, consented remote request.

Further evidence should remain finite and claim-driven. The native x86_64 CI result closes only the bounded equality check for the named hosted jobs. Independent cross-machine reproduction would address same-project and hosted-environment bias. Frozen adversarial ledgers for both readers and the temporary remote boundary would address only enumerated path, graph, disclosure, event, and cleanup properties. A later human study would require a new protocol and would be necessary before any efficacy claim.

9. Reproducibility and Availability

The public source repository is available at:

`https://github.com/mehmetsolakedu/activity-radar`

The historical source tag and CI record are:

- Release `v1.2.0-beta.2`: https://github.com/mehmetsolakedu/activity-radar/releases/tag/v1.2.0-beta.2
- GitHub Actions run `32648392604`: https://github.com/mehmetsolakedu/activity-radar/actions/runs/32648392604

The protocol, runner, result, erratum, hardening, and manuscript source are maintained on branch `codex/aiwingman-technical-preprint`. The exact software checkpoint evaluated by the cited CI run is `99faac3...`; later manuscript-package commits do not retroactively become evidence from that run:

`https://github.com/mehmetsolakedu/activity-radar/tree/codex/aiwingman-technical-preprint`

The latest evaluated source checkpoint represented in the current evidence package is `99faac3ef52d0da72d082706f64903a6aacd2c6d`, with exact-commit CI record:

`https://github.com/mehmetsolakedu/activity-radar/actions/runs/32659669054`

These content-addressed Git commits establish repository identity, not independent archival preservation. The manuscript source and generated documents may receive later content-only revisions before deposit; no preprint DOI or archival release is claimed.

Reproduction should begin from a separate detached worktree at the exact runner revision:

```
git worktree add --detach ../aiwingman-fa5186e \
  fa5186e5d04fedfd75daac72e533a1daf9dbfa89
cd ../aiwingman-fa5186e
git rev-parse HEAD
git status --porcelain
shasum -a 256 \
  Package.swift \
  Sources/ActivityRadarCore/WorkContinuity.swift \
  Sources/AiWingmanResearchBenchmark/main.swift \
  Research/protocol/SPECIFICATION_V1.md \
  Research/generate_continuity_corpus.py \
  Research/fixtures/continuity-policy-corpus-v1.json
```

The `git rev-parse HEAD` command must report `fa5186e5d04fedfd75daac72e533a1daf9dbfa89`, and `git status --porcelain` must be empty. In the listed order, the expected SHA-256 values are `03389531d1cd8aea9222e2663a603b89fbacbba1fa48e4813d2e415dae107425`, `ff68e565bb1cebff47d66a23488e84b4e7bc2bf39cf8b75740683a7a055fa895`, `068f085273f8a7638c7a0e05d13b32f0f39cc3ed519bd2327b4ef5551417906b`, `ee6ab6568a3fcf8332facf9ef0a2d3bae66ab15d7f09a63ca6cbef6ce44e6f11`, `2c27f2bd7950e0aa26b8878ae27e13dfcd3332571f32b61444feb1111a8aae3c`, and `183025163326aeb7f95470d2ecf494b31bc8239c8abb3e74e939967cf3ad8f30`. These checks establish source identity before building; the V1 result files themselves did not attest executable provenance.

The separate Python oracle can regenerate the frozen expected-output corpus for a byte comparison without overwriting the committed fixture:

```
corpus_check="$(mktemp)"
trap 'rm -f "$corpus_check"' EXIT
python3 Research/generate_continuity_corpus.py \
  --spec Research/protocol/SPECIFICATION_V1.md \
  --output "$corpus_check"
cmp Research/fixtures/continuity-policy-corpus-v1.json "$corpus_check"
shasum -a 256 "$corpus_check"
```

Successful regeneration must be byte-identical and report SHA-256 `183025163326aeb7f95470d2ecf494b31bc8239c8abb3e74e939967cf3ad8f30`. This verifies deterministic corpus generation from the named same-project oracle and specification; it does not convert that oracle into an external reference standard.

The primary benchmark command is:

```
swift run -c release AiWingmanResearchBenchmark \
  --corpus Research/fixtures/continuity-policy-corpus-v1.json \
  --output Research/results/continuity-benchmark-reproduction.json
```

The post-freeze fresh-process driver first appears at result-package commit `e46686588793173d1b299b1829a92dbab7e9528d`, not at the runner revision. Add a second detached worktree, verify that driver, and invoke it against the executable and corpus in the clean runner worktree. The `--runner-commit` argument becomes operator-supplied metadata in the supplementary summary; it does not attest the executable's provenance:

```
git worktree add --detach ../aiwingman-e466865 \
  e46686588793173d1b299b1829a92dbab7e9528d
git -C ../aiwingman-e466865 rev-parse HEAD
git -C ../aiwingman-e466865 status --porcelain
shasum -a 256 \
  ../aiwingman-e466865/Research/run_cross_process_determinism.py

swift build -c release --product AiWingmanResearchBenchmark

python3 ../aiwingman-e466865/Research/run_cross_process_determinism.py \
  --executable "$PWD/.build/release/AiWingmanResearchBenchmark" \
  --corpus "$PWD/Research/fixtures/continuity-policy-corpus-v1.json" \
  --output "$PWD/cross-process-determinism-reproduction.json" \
  --runner-commit fa5186e5d04fedfd75daac72e533a1daf9dbfa89 \
  --processes 100 \
  --work-directory "$PWD"
```

The second `git rev-parse HEAD` command must report `e46686588793173d1b299b1829a92dbab7e9528d`, its status output must be empty, and the driver SHA-256 must be `fffcc2390d0b75a48c424bda6414055a00f06f50cb54e7ad61150f5b5f1b9070`.

The reproduction target is 155/155 exact conformance and the canonical corpus digest `fd72e52b8098d2f62f6c6f7ccc6de7e744de7ed03722d08c7f983527601db8e6`, optionally followed by the post-freeze supplementary check of 100/100 fresh processes returning that digest. A new full-report JSON is not expected to reproduce the archived report SHA-256 because timestamps, performance measurements, and per-run report hashes can differ. The archived result-manifest hashes establish byte equality relative to the committed manifest; they do not authenticate execution or prove provenance. The repository includes versioned citation metadata in keeping with software-citation principles [32]. Software source, tests, scripts, and executable tooling are MIT-licensed. Effective 24 August 2026, the manuscript, generated scholarly outputs, written research protocol, synthetic fixture corpus, result data, manifests, summaries, and errata identified in `paper/LICENSE_STATUS.md` are licensed under Creative Commons Attribution 4.0 International. Private Codex databases, rollout files, task text, local paths, credentials, real-work screenshots, and third-party works are outside the publication package and license grant.

9.1 Manuscript artifact build

The submission-manuscript build environment used Python 3.12.13 with the package versions pinned in `paper/requirements.txt`. A clean environment can build the editable Word manuscript and author-rendered reference PDF as follows:

```
python3 -m venv .venv-paper
. .venv-paper/bin/activate
python -m pip install -r paper/requirements.txt
python paper/build_submission_docx.py \
  --source paper/aiwingman_technical_report.md \
  --output output/docx/aiwingman-technical-note-v1.docx
python paper/build_preprint.py \
  --source paper/aiwingman_technical_report.md \
  --output output/pdf/aiwingman-technical-note-v1.pdf
```

The PDF builder selects macOS Times New Roman and Arial when available and otherwise uses the bundled ReportLab Vera faces. The DOCX declares Calibri for document text and rasterizes its two generated diagrams with Arial or the same Vera fallback. Word/LibreOffice version, installed fonts, PDF metadata, and ZIP timestamps can change bytes or pagination; byte-identical manuscript regeneration is therefore not claimed across environments. The final deposit bytes must be rendered page by page, scanned structurally and for compressed private content, and identified by SHA-256 values in the final publication artifact manifest before upload.

10. Claim Ledger

Claim	Status in this revision	Exact boundary
AiWingman is an open-source Codex-specific retrospective overlay	Supported as a project description	Third-party community software; not an official OpenAI product or stable API integration
Ordinary dashboard and optional Wingman use separate readers	Supported by source architecture	Ordinary path is per-candidate-row; tree aggregation and graph signals belong only to Wingman
Codex source state is queried through logical read-only adapters	Implementation property under engineering review	No application SQL write to database content; WAL-mode read-only opening may depend on existing readable `-wal`/`-shm`, a writable containing directory, or immutable mode; application writes its own state; no whole-application write-free claim
Pure Swift continuity policy conforms to the frozen corpus	155/155 passed on recorded arm64 environment	Exact agreement with the finite same-project expected outputs; four triage reason/weight rules and the zero-input boundary are not exposed in V1
Post-freeze focused tests expose the enumerated V1 observability gaps	6/6 focused tests passed and 6/6 simple corresponding mutations were killed on the current arm64 working revision	Same-project supplementary regression only; not part of V1, exhaustive coverage, or human-utility evidence
Local policy output is repeatable	Across 100 evaluations per fixture, evaluations 2-100 matched evaluation 1 (15,500 evaluations; 15,345 nontrivial comparisons); 100/100 post-freeze fresh processes returned the same corpus digest	Fresh-process check was supplementary; archived `evaluationCount` is an erratum, not a function-call denominator; core result is one arm64 machine; excludes readers, UI, graphs, and remote output
Five exact seeded strings did not propagate to the scanned policy output surface	Zero occurrences for five sentinels over 155 scanned outputs	Historical `contentNeutrality` label is an erratum; outputs retain identifiers; not semantic independence, anonymity, privacy, noninterference, or secret-removal proof
Confirmed obsolete/abandoned requires user confirmation	Zero violations over 70 unconfirmed fixtures	Synthetic lifecycle contract only; not an audit of every application path
Technical contrasts change frozen no-recommendation decisions	Recency-only selected an item in 35 cases; same-score/no-suppression selected one in 16	Recency-only also removes deferrals and eligibility; only the 16-case contrast isolates threshold and lead removal; no quality or superiority inference
Observed median policy latency was sub-millisecond at 1,000 synthetic inputs on the measured Mac	Median 0.511 ms triage and 0.388 ms lifecycle	Normalized synthetic policy inputs on one named environment only

Claim	Status in this revision	Exact boundary
Historical dual-architecture release regression evidence	52 tests and 16 self-tests passed per architecture at `5e212181...`	Older release commit; not current policy-result or revision branch
Wingman tree token quantity measures exact cost or waste	Not supported	Maximum cumulative tree counter is only a comparison proxy
Graph signals measure importance or success	Not supported	Descriptive and unevaluated
Runner revision and clean-worktree provenance are machine-attested by V1 results	Not supported	Clean detached execution is author-reported; reproduction must verify HEAD, empty status, and source hashes before building
Optional remote review is security-isolated, residue-free under crashes, or sole-context	Not supported	Preview, a process-wide gate, two bounded temporary prefixes, and verified normal/error cleanup narrow intended disclosure; they do not prove isolation, crash cleanup, or service retention
Human efficacy or productivity benefit	Untested	No participants or outcome measures
Comparative superiority or first-of-kind status	Not claimed	Prior research and current products provide overlapping capabilities
Hardened-source hosted arm64/x86_64 policy equality and public-source preparation check	Passed at `99faac3...`, run `32659669054`	Post-freeze engineering evidence for the named jobs; not external reproduction, broad determinism, independent archiving, a signed public release, or macOS 13 runtime evidence

11. Declarations

Ethics and data statement. This report describes software architecture and a synthetic engineering evaluation. The reported evaluation used only synthetic fixtures; it recruited no participants and did not export or analyze private task histories as research data. It involved no animal, plant, survey, or interview data. No human-outcome inference is made.

Funding. This research received no external funding.

Author contribution. Mehmet Solak conceived the product direction and study, defined the intended use and policy, directed and reviewed iterative software and research-package development, verified the reported claims, references, results, and artifacts, wrote and revised the manuscript with the disclosed AI assistance, and is the sole author. Mehmet Solak accepts full responsibility for the work.

AI assistance. OpenAI Codex assisted with source discovery, code inspection, literature organization, drafting and revising the specification, Python oracle, synthetic fixtures, benchmark runner, tests, manuscript, document formatting, and author-directed internal adversarial reviews of claims, methods, security boundaries, and publication artifacts. Those same-project reviews were not human or external peer review and were not independent validation. The named author has reviewed and verified every claim, reference, result, and generated artifact, revised the prose as needed, and accepts full responsibility. AI is not listed as an author.

Software and research-package licenses. Executable software source, tests, scripts, build tooling, and research drivers are distributed under the MIT License. Effective 24 August 2026, the manuscript, generated scholarly outputs, written research protocol, synthetic fixture corpus, result data, manifests, summaries, and errata identified in `paper/LICENSE_STATUS.md` are distributed under Creative Commons Attribution 4.0 International. The author confirms the right to grant both license scopes.

Identity and correspondence. The named author confirms the public name form “Mehmet Solak,” ownership of ORCID `0000-0002-0800-0334`, the Siirt University Biosystems Engineering affiliation, and `mehmetsolak@siirt.edu.tr` as the correspondence address. The same identity appears in the manuscript, document metadata, submission metadata, and citation file.

Corresponding-author responsibility. Mehmet Solak accepts responsibility for answering questions or comments about the preprint and for providing the reported data or materials when reasonably requested and legally permitted.

Data Availability Statement

The protocol, synthetic fixtures, benchmark runner, result artifacts, and source supporting this report are publicly available at the repository and exact checkpoints listed in Section 9. The non-executable protocol, synthetic fixtures, and archived results are also packaged in the CC BY 4.0 research supplement accompanying this submission. The reported evaluation used no private Codex task histories, and none are included in the publication package.

Conflicts of Interest

The author is the creator and maintainer of AiWingman, the open-source software evaluated in this manuscript; this relationship is disclosed. The author declares no other financial or non-financial competing interests.

References

[1] M. Kersten and G. C. Murphy, “Using Task Context to Improve Programmer Productivity,” Proceedings of the 14th ACM SIGSOFT International Symposium on Foundations of Software Engineering, pp. 1-11, 2006. https://doi.org/10.1145/1181775.1181777
[2] C. Parnin and R. DeLine, “Evaluating Cues for Resuming Interrupted Programming Tasks,” Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, pp. 93-102, 2010. https://doi.org/10.1145/1753326.1753342
[3] C. Parnin and S. Rugaber, “Resumption Strategies for Interrupted Programming Tasks,” Software Quality Journal, vol. 19, no. 1, pp. 5-34, 2011. https://doi.org/10.1007/s11219-010-9104-9
[4] M.-A. Storey, J. Ryall, J. Singer, D. Myers, L.-T. Cheng, and M. Muller, “How Software Developers Use Tagging to Support Reminding and Refinding,” IEEE Transactions on Software Engineering, vol. 35, no. 4, pp. 470-483, 2009. https://doi.org/10.1109/TSE.2009.15
[5] C. Treude and M.-A. Storey, “Awareness 2.0: Staying Aware of Projects, Developers and Tasks Using Dashboards and Feeds,” Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering—Volume 1, pp. 365-374, 2010. https://doi.org/10.1145/1806799.1806854
[6] O. Baysal, R. Holmes, and M. W. Godfrey, “No Issue Left Behind: Reducing Information Overload in Issue Tracking,” Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, pp. 666-677, 2014. https://doi.org/10.1145/2635868.2635887
[7] V. Dibia et al., “AutoGen Studio: A No-Code Developer Tool for Building and Debugging Multi-Agent Systems,” Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 72-79, 2024. https://doi.org/10.18653/v1/2024.emnlp-demo.8
[8] D. Gao et al., “AgentScope: A Flexible yet Robust Multi-Agent Platform,” arXiv:2402.14034v2, 2024. https://arxiv.org/abs/2402.14034v2
[9] C. Ma, J. Zhang, Z. Zhu, C. Yang, Y. Yang, Y. Jin, Z. Lan, L. Kong, and J. He, “AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents,” Advances in Neural Information Processing Systems, vol. 37, Datasets and Benchmarks Track, pp. 74325-74362, 2024. https://doi.org/10.52202/079017-2365
[10] L. Dong, Q. Lu, and L. Zhu, “AgentOps: Enabling Observability of LLM Agents,” arXiv:2411.05285v2, 2024. https://arxiv.org/abs/2411.05285v2
[11] A. AlSayyad, K. Y. Huang, and R. Pal, “AgentTrace: A Structured Logging Framework for Agent System Observability,” arXiv:2602.10133v1, 2026. https://arxiv.org/abs/2602.10133v1
[12] B. Kitano, E. Carlson, B. Russett, and A. Kesling, “Managing Multi-Agent Research Systems: A Dashboard for Human Oversight of Coordinating AI Agents,” HEAL@CHI 2026 workshop paper; workshop-hosted PDF, 4 pp., 2026, accessed 23 August 2026. https://heal-workshop.github.io/chi2026_papers/Managing%20Multi-Agent%20Research%20Systems%20A%20Dashboard%20for%20Human%20Oversight%20of%20Coordin.pdf
[13] S. Dhanorkar, S. Passi, and M. Vorvoreanu, “Human Oversight of Agentic Systems in Practice: Examining the Oversight Work, Challenges, and Heuristics of Developers Using Software Agents,” arXiv:2606.05391v1, 2026. https://arxiv.org/abs/2606.05391v1
[14] A. Zhu, L. Dugan, and C. Callison-Burch, “ReDel: A Toolkit for LLM-Powered Recursive Multi-Agent Systems,” Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 162-171, 2024. https://doi.org/10.18653/v1/2024.emnlp-demo.17
[15] W. Epperson et al., “Interactive Debugging and Steering of Multi-Agent AI Systems,” Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, pp. 1-15, 2025. https://doi.org/10.1145/3706598.3713581
[16] M. Desmond et al., “Agent Trajectory Explorer: Visualizing and Providing Feedback on Agent Trajectories,” Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, no. 28, pp. 29634-29636, 2025. https://doi.org/10.1609/aaai.v39i28.35350
[17] J. Wang et al., “Illuminating LLM Coding Agents: Visual Analytics for Deeper Understanding and Enhancement,” arXiv:2508.12555v1, 2025. https://arxiv.org/abs/2508.12555v1
[18] T. Ou, W. Guo, A. Gandhi, G. Neubig, and X. Yue, “AgentDiagnose: An Open Toolkit for Diagnosing LLM Agent Trajectories,” Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 207-215, 2025. https://doi.org/10.18653/v1/2025.emnlp-demos.15
[19] T. Gao et al., “Graph of Trace: Visualizing Execution Traces of Scientific Agents,” Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations), pp. 297-306, 2026. https://doi.org/10.18653/v1/2026.acl-demo.29
[20] GitHub, “Managing Agent Sessions,” GitHub Copilot documentation. Accessed 23 August 2026. https://docs.github.com/en/copilot/how-tos/copilot-on-github/use-copilot-agents/manage-and-track-agents
[21] GitHub, “About GitHub Copilot CLI Session Data,” GitHub Copilot documentation. Accessed 23 August 2026. https://docs.github.com/en/copilot/concepts/agents/copilot-cli/chronicle
[22] GitHub, “Working with Agent Sessions in the GitHub Copilot App,” GitHub Copilot documentation. Accessed 23 August 2026. https://docs.github.com/en/copilot/how-tos/github-copilot-app/agent-sessions
[23] OpenAI, “Notifications,” official OpenAI documentation. Accessed 24 August 2026. https://learn.chatgpt.com/docs/notifications
[24] OpenAI, “Long-Running Work,” official OpenAI documentation. Accessed 24 August 2026. https://learn.chatgpt.com/docs/long-running-work
[25] OpenAI, “Codex App Server,” official OpenAI documentation. Accessed 24 August 2026. https://learn.chatgpt.com/docs/app-server
[26] OpenAI, “Code Review,” official OpenAI documentation. Accessed 24 August 2026. https://learn.chatgpt.com/docs/code-review
[27] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-First Software: You Own Your Data, in Spite of the Cloud,” Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, pp. 154-178, 2019. https://doi.org/10.1145/3359591.3359737
[28] A. Cooper, H. Tschofenig, B. Aboba, J. Peterson, J. Morris, M. Hansen, and R. Smith, “Privacy Considerations for Internet Protocols,” RFC 6973, Internet Architecture Board, July 2013. https://doi.org/10.17487/RFC6973
[29] C. K. Chow, “On Optimum Recognition Error and Reject Tradeoff,” IEEE Transactions on Information Theory, vol. 16, no. 1, pp. 41-46, 1970. https://doi.org/10.1109/TIT.1970.1054406
[30] R. El-Yaniv and Y. Wiener, “On the Foundations of Noise-free Selective Classification,” Journal of Machine Learning Research, vol. 11, no. 53, pp. 1605-1641, 2010. https://www.jmlr.org/papers/v11/el-yaniv10a.html
[31] SQLite, “Write-Ahead Logging,” SQLite Documentation. Accessed 23 August 2026. https://www.sqlite.org/wal.html
[32] A. M. Smith, D. S. Katz, K. E. Niemeyer, and FORCE11 Software Citation Working Group, “Software Citation Principles,” PeerJ Computer Science, vol. 2, e86, 2016. https://doi.org/10.7717/peerj-cs.86